TEXAS A&M UNIVERSITY

Algorithms and Their Complexity

Discrete Structures for Computing

Algorithm

An algorithm is a finite sequence of steps that solves a problem.

Computational complexity

How much computing resources are needed to solve a problem?

How long (time) and how much memory (space) does it take?

We observe the behavior of algorithms as the input size grows.

Algorithm

We are interested in:

- The behavior of algorithms as the input size grows.
- Modeling a problem and an algorithm mathematically
- Analyzing the growth of functions (as argument grows without bound)
- The complexity of algorithms

Modeling a Problem

Translate a real-world problem into a mathematical context.

e.g.
Sorting a set of social security numbers and sorting a set of temperatures are both special cases of sorting a set of elements from a totally ordered set.

Algorithms

Algorithms can be described in English or in pseudocode.

Pseudocode is an intermediate step between an English description of the steps and the implementation of these steps in code.

Advantages of pseudocode:

- Independent of the programming language
- More general than a specific programming language

Properties of algorithms

Input: An algorithm has input values from a specified set.

Output: An algorithm produces output values from a specified set. The output values are the **solution**.

Definiteness: The steps of an algorithm must be properly defined.

Correctness: An algorithm should produce the **correct** output values for each set of input values.

Finiteness: An algorithm should produce the output after a **finite** number of steps for any input.

Effectiveness: It must be possible to perform each step of the algorithm correctly and in **a finite amount of time**.

Generality: The algorithm should work **for all** problems of the desired form.

Maximum-Finding Algorithm

Find the maximum of sequence: $a_0, a_1, a_2, \ldots, a_{n-1}$:

```
max ← a_0
for i ← 1...(n-1) do
   if max < a_i then
      max ← a_i
   end if
end for
return max
```

Activity

Does the maximum-finding algorithm have the desired properties of an algorithm?

The Searching Problem

Input: list of elements $a_0, a_1, a_2, \ldots, a_{n-1}$ and a particular element x

Output: return the index in the list where x appears; if x is not in the list then return –1

For now, assume all the list elements are unique.

Linear Search Algorithm

Input: $a_0, a_1, a_2, \ldots, a_{n-1}$ and x
Output: index of x or –1

```
i ← 0
while i < n and x ≠ a_i do
    i ← i + 1
end while
if i < n then
    return i
else
    return -1
end if
```

Linear Search Algorithm

Linear search can be slow

If x is not in the list or is toward the end we check all (or most) elements in the list.

We can see that the running time is proportional to the number of elements in the list

Binary Search Example

Search for the number **19** in the list:

1 2 3 5 6 7 8 10 12 13 15 16 18 19 20 22

Compare 19 with the element roughly in the middle

1 2 3 5 6 7 8 **10** 12 13 15 16 18 19 20 22

Since 19 > 10, ignore the first half of the list

… 12 13 15 **16** 18 19 20 22

Since 19 > 16, ignore the first half of the list

… 18 **19** 20 22 **Found!**

Binary Search Algorithm

Input: a sorted list stored in array A[0 . . . n–1] and x

Output: index of x or –1

```
1:
2:
3:
4:
5:
6:
7:
8:
9:
10:
11:
12:
```

Exponent vs. Logarithm

Exponent: how many times a number is used in a multiplication.

$2 \times 2 \times 2 = 8 = 2^3$

exponent

base

Logarithm: how many of one number to multiply to get another number.

$\log_2(8) = 3$

exponent

$2^3 = 8 \Leftrightarrow \log_2(8) = 3$

base

$\log_a(a^b) = b$

$y = a^x$

x y

$x = \log_a(y)$

Binary Search Algorithm

Let n be the size of the list

Every iteration of the while loop reduces the size of the search interval by a factor of 2

After about $\log_2 n$ iterations, the search interval consists of just a single list element

The running time is proportional to the logarithm of the size of the list

The Sorting Problem

Input: list of elements $a_0, a_1, a_2, \ldots, a_{n-1}$ drawn from totally ordered set (supports $<$ operation)

Output: list of elements $b_0, b_1, b_2, \ldots, b_n$ that is a rearrangement of the input list such that $b_0 < b_2 < \ldots < b_{n-1}$.

Assume all the list elements are unique.

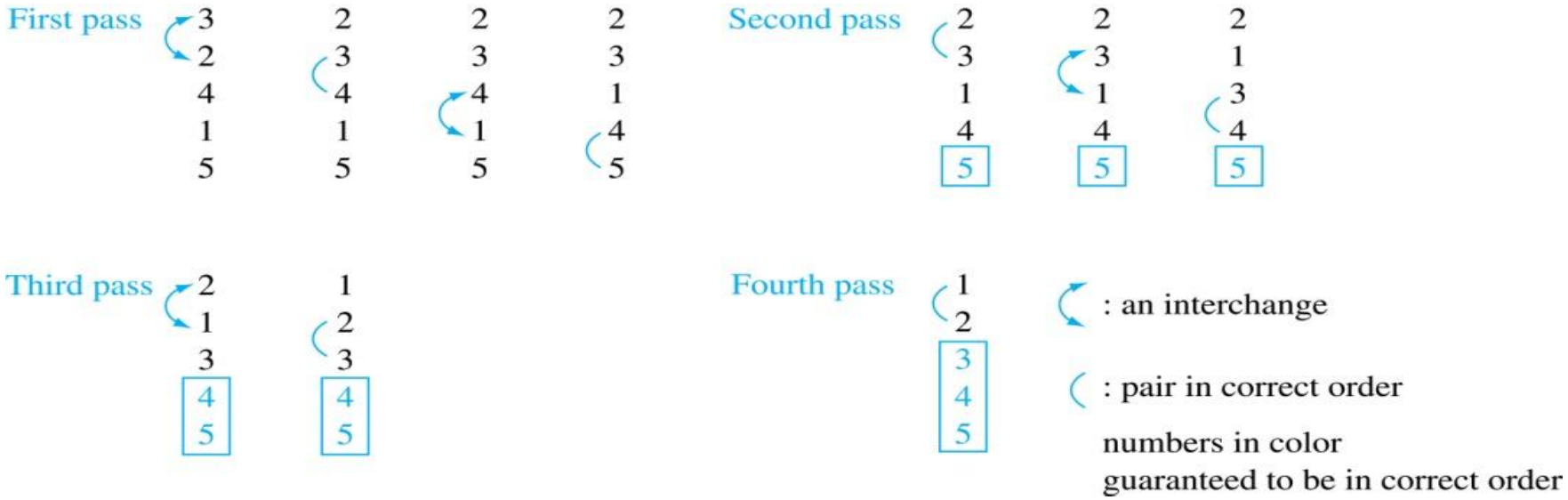
Bubble Sort Algorithm

Bubble sort makes multiple passes through a list.

Every pair of elements that are found to be out of order are interchanged.

Bubble Sort

e.g. Steps of bubble sort with: {3, 2, 4, 1, 5}



- After the 1st pass the largest element is in the correct position
- After the 2nd pass, the 2nd largest element is in the correct position
- After each subsequent pass, an additional element is put in the correct position

Bubble Sort Algorithm

Input: array A[0 ..n] of elements

Output: array A[0 ..n] of sorted elements

```
for i:=0 to n-2
  for j:=0 to n-i-1
    if a_j > a_j+1 then interchange a_j and a_j+1
  end for
end for

{a_1,..., a_n is now in increasing order}
```

Insertion Sort Algorithm

Assume the input list is provided in an array.

We repeatedly increase the prefix of the array that is in sorted order.

1. Take the next element x from the not-yet-sorted suffix of the array.
2. Find the correct location for x in the sorted prefix of the array.
3. Shift to make room for x in the sorted prefix.
4. Insert x at its correct position.

Insertion Sort Algorithm

Sort: 7 3 1 2 4 8

#1	7	3	1	2	4	8
#2	**7**	3	1	2	4	8
#3	**3**	7	1	2	4	8
#4	3	1	7	2	4	8
#5	**1**	3	7	2	4	8
#6	1	3	2	7	4	8
#7	1	**2**	3	7	4	8
#8	1	2	3	4	7	8
#9	1	2	3	**4**	7	8
#10	1	2	3	4	7	**8**

Input: array A[0 ..n–1] of elements
Output: array A[0 ..n–1] of sorted elements

```
for j ← 1...n-1 do // j is index where unsorted suffix starts
  m ← A[j]       // A[j] holds next element to insert
  i ← 0
  while (m > A[i]) // Use linear search to find m's place
     i ← i+1        // i is the index where m should go
   end while
  for k ← j...i+1 do
    A[k] := A[k - 1] // shift places to make room for m
  end for
  A[i] ← m           // Insert m in its place
end for
return A
```

Activity 4.1

Sort a list of numbers using insertion sort

Original list: 45 23 26 44 55 32 10 84

Use one row to show the list after each pass through the algorithm.

Activity 4.1 solution

1	45 23 26 44 55 32 10 84
2	**23** 45 26 44 55 32 10 84
3	23 **26** 45 44 55 32 10 84
4	23 26 **44** 45 55 32 10 84
5	23 26 44 45 **55** 32 10 84
6	23 26 **32** 44 45 55 10 84
7	**10** 23 26 32 44 45 55 84
8	10 23 26 32 44 45 55 **84**

Note: the numbers swapped during the insertion are not displayed.

Insertion Sort Algorithm

The runtime depends on the initial order of the array, and how much shifting is required

Soon we will learn techniques for showing that the runtime is proportional to n^2 in the worst case.

Estimating Algorithm Runtime

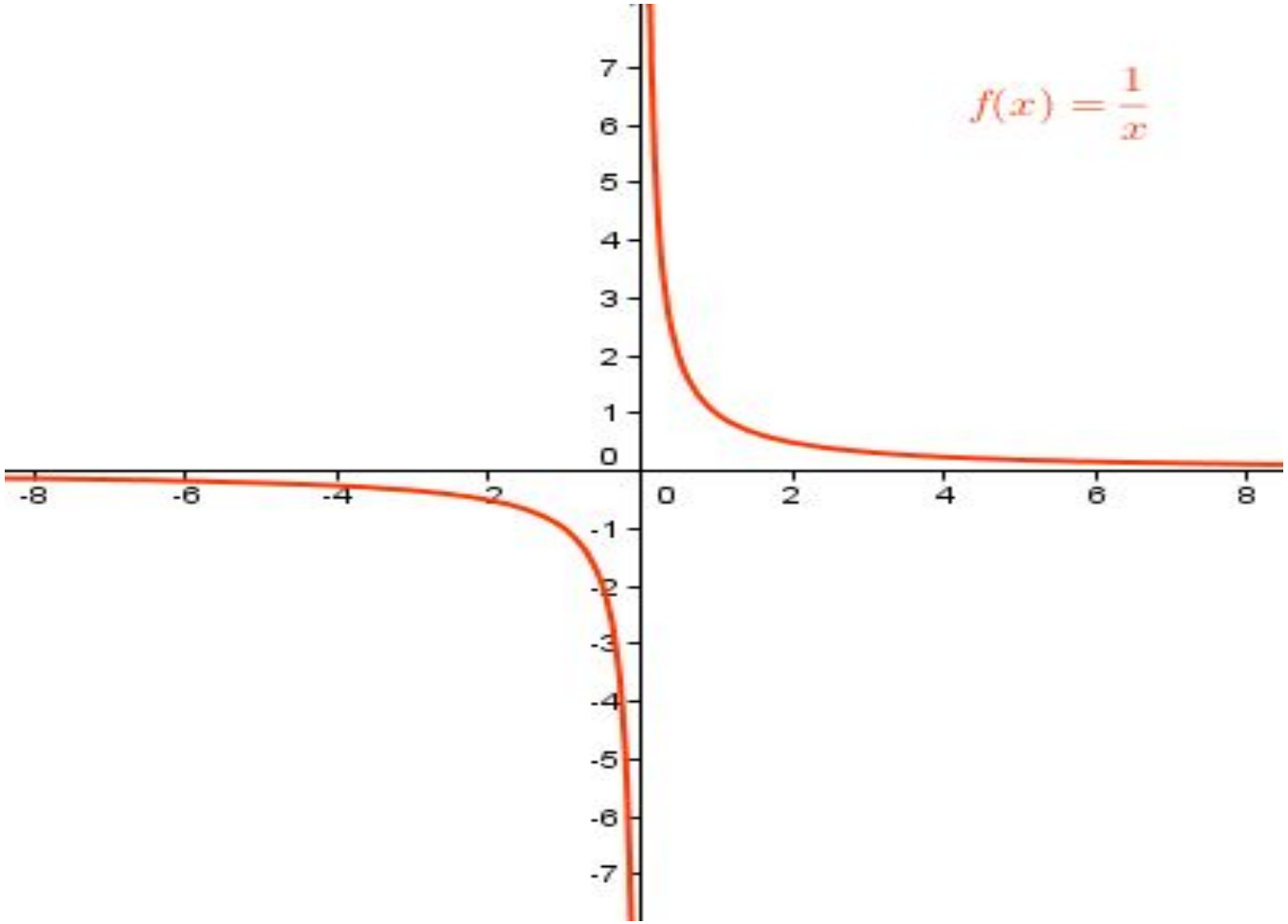
How to describe runtime?
As a function of the input size

Different inputs, even of the same length, might take different amounts of time.

The runtime depends on:

1. The programming language
2. The compiler
3. The hardware
4. The level of multiprogramming

Asymptotic Analysis



Asymptotic Analysis

Informally, measure time as a function of input size:

1. ignore multiplicative constants
2. ignore lower order terms

Focus on the behavior as the input size grows to infinity.

As the input size increases, the impact of multiplicative constants and lower order terms becomes negligible!

Big-*O* Notation O()

Big-O is how to do the asymptotic analysis.

Definition: Let f and g be functions.

f(x) is O(g(x)) if there are constants C and k such that:

$$|f(x)| \leq C\,|g(x)| \text{ whenever } x > k$$

This is read as "f(x) is big-O of g(x)"

C and k are the witnesses.

Big-O Estimates (Witness C)

To find the witness C for an upper bound g(x)

1. Add the absolute values of all the coefficients

or

2. Add the positive coefficients only.

Example: $f(x) = 5x^2 - 3x + 20$

Method 1: $5x^2 - 3x + 20 \leq 5x^2 + 3x + 20 \leq 5x^2 + 3x^2 + 20x^2 \rightarrow C = 28.$

Method 2: $5x^2 - 3x + 20 \leq 5x^2 + 20 \leq 5x^2 + 20x^2 \rightarrow C = 25.$

Big-*O* Estimates for Polynomials

Let $$f(x) = a_n x^n + a_{n-1} x^{n-1} + \cdots + a_1 x + a_o$$

where a_0, a_1, … a_n are real numbers with $a_n \neq 0$.

The leading term $a_n x^n$ of a polynomial dominates the growth.

Therefore $f(x)$ is $O(x^n)$

Big-*O* Example 1

Show that $f(x) = x^2 + 2x + 1$ is $O(x^2)$

Find a positive C and k such that:

$x^2 + 2x + 1 \leq C.x^2$ for all $x > k$

x^2 increases faster than $2x + 1$

Let's try using four x^2 terms, i.e. $C = 4$

How big must x be to ensure that $x^2 + 2x + 1 \leq 4x^2$?

$x > 1$, $k = 1$

Big-*O* Example 2

Suppose that an algorithm, running on an input of size n, takes $4n^2 + 1000n + 904$ instructions.

As n grows which term dominates the growth?

Why?

What is g(n) such that f(n) is big-O of g(n)?

Big-*O* Example 2 solution

Suppose that an algorithm, running on an input of size n, takes $4n^2 + 1000n + 904$ instructions.

As n grows which term dominates the growth? $4n^2$

Why? $4n^2 + 1000n + 904$ is a quadratic equation, and $4n^2$ is the term that has the highest degree.

What is g(n) such that f(n) is big-O of g(n)?

$4n^2 + 1000n + 904 \leq (4+1000+904)\, n^2 = 1908\, n^2$ for $n \geq 1$

$\therefore$ $4n^2 + 1000n + 904$ is big-O of n^2 with the witnesses (C=1908, k=1)

Big-*O* Example 3 Example

Suppose that an algorithm, running on an input of size n, takes $8n^3 - 6n^2 - 12n + 23$ instructions.

As n grows which term dominates the growth?
Why?
What is g(n) such that f(n) is big-O of g(n)?

Big-*O* Example 3 solution

Suppose that an algorithm, running on an input of size n, takes $8n^3 - 6n^2 - 12n + 23$ instructions.

As n grows which term dominates the growth? $8n^3$

Why? $8n^3 - 6n^2 - 12n + 23$ is a polynomial, and $8n^3$ is the term that has the highest degree.

What is g(n) such that f(n) is big-O of g(n)?

$8n^3 - 6n^2 - 12n + 23 \leq (8 + 6 + 12 + 23)n^2 = 49\ n^2$ for $n \geq 1$

$\therefore$ $8n^3 - 6n^2 - 12n + 23$ is big-O of n^3 with the witnesses (C=49, k=1)

Big-*O* Example

Show that n^2 is not O(n).

We have to show that all pairs of C and k fail.

Let's try a proof by contradiction:

Suppose there is C and k | $n^2 \leq C.n$ for all $n > k$.

Divide both sides by n to get: $n \leq C$ for all $n > k$.

C is a constant and eventually some value of n exceeds C

This is a contradiction!

Proof of C for Big-*O*

Let $f(x)= a_n x^n + \ldots + a_1 x + a_0$. Then f(x) is $O(x^n)$

Proof:

$$|f(x)| = |a_n x^n + a_{n-1} x^{n-1} + \ldots + a_1 x + a_0|$$

$$|f(x)| \leq |a_n| x^n + |a_{n-1}| x^{n-1} + \ldots + |a_1| x + |a_0|$$

$$|f(x)| \leq x^n \left(|a_n| + \frac{|a_{n-1}|}{x} + \ldots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n}\right)$$

$$x^n \left(|a_n| + \frac{|a_{n-1}|}{x} + \ldots + \frac{|a_1|}{x^{n-1}} + \frac{|a_0|}{x^n}\right) \leq x^n \left(|a_n| + |a_{n-1}| + \ldots + |a_1| + |a_0|\right)$$

$|f(x)| \leq x^n (|a_n| + |a_{n-1}| + \ldots + |a_1| + |a_0|)$ as long as $x > 1$

$k = 1$ and $C = |a_n| + |a_{n-1}| + \ldots |a_1| + |a_0|$

Big-*O* Examples

1. Sum of first n positive integers:

$1 + 2 + 3 + \ldots + n \leq n + n + n + \ldots n \leq n^2$, which is $O(n^2)$

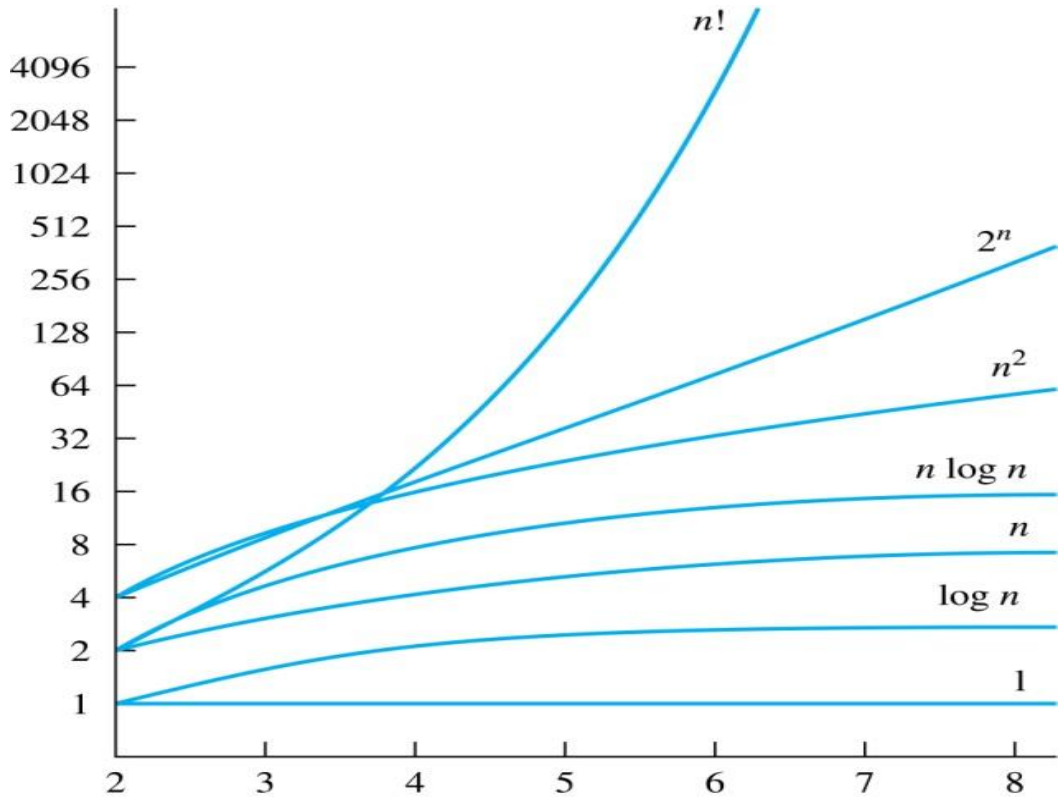
2. Factorial function:

$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n \leq n \cdot n \cdot n \cdots n \cdot n = n^n$, which is $O(n^n)$

3. Logarithm (base 2) of factorial:

$\log(n!) \leq \log(n^n) = n \log n$, which is $O(n \log n)$

Growth of Functions



Note the difference in behavior of functions as n gets larger

Combinations of Functions

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then:

$$(f_1 + f_2)(x) \text{ is } O(\max(|g_1(x)|, |g_2(x)|))$$

If $f_1(x)$ and $f_2(x)$ are both $O(g(x))$ then:

$$(f_1 + f_2)(x) \text{ is } O(g(x))$$

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then:

$$(f_1 \cdot f_2)(x) \text{ is } O(g_1(x) \cdot g_2(x))$$

Big-*O* Examples

What are the Big-O estimates for these functions?

$f(n) = (n^2 + 8)(n + 1)$

$f(n) = (n! + 2^n)\,(n^3 + \log(n^2 + 1))$

Big-O examples solution

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then:

$$(f_1 + f_2)(x) \text{ is } O(\max(|g_1(x)|, |g_2(x)|))$$

If $f_1(x)$ is $O(g_1(x))$ and $f_2(x)$ is $O(g_2(x))$ then:

$$(f_1 \cdot f_2)(x) \text{ is } O(g_1(x) \cdot g_2(x))$$

Therefore:

$f(n) = (n^2 + 8)(n + 1)$ is $O(n^2 \times n) = O(n^3)$

$f(n) = (n! + 2^n)(n^3 + \log(n^2 + 1))$ is $O(n! \times n^3)$

Practice

What is the Big-O estimate for this function?

$f(n) = 9n^2 + 8\ n \log(n)$

Practice

What is the Big-O estimate for this function?

$f(n) = (n^3+1) \log(n^2 + 1) + 3n^2$

The Complexity of Algorithms

How efficient is an algorithm for a given input size?

How much time does an algorithm use to solve a problem?

How much computer memory does an algorithm use to solve a problem?

When we analyze the time an algorithm uses, we study the time complexity of the algorithm. ⏱️

When we analyze the computer memory the algorithm uses, we study the space complexity of the algorithm.

The Complexity of Algorithms

Let's use the last two topics: Algorithms and Asymptotic Analysis, to analyze the time complexity of algorithms.

We will measure the time complexity as the number of operations executed.

Simplifying Assumption:
All operations take the same amount of time.

Why?

Operations might vary by a constant.

(e.g., a division might take 4 times longer than an addition on some hardware), but we ignore constant factors.

Operations count

How to estimate the number of operations used by instructions.

Time Complexity
Maximum Finding Algorithm: O(n)

Total is at most $1 + 3n + n + n + 1 = 5n + 2 = O(n)$

1.	max $\leftarrow a_0$	1: 'assign' (1 op)
2.	**for** i $\leftarrow$ 1...n **do**	2: 'add', 'assign', 'test' n times (3n + 1 ops)
3.	**if** max < a_i **then**	3: 1 'compare' per iteration (n ops)
4.	max $\leftarrow a_i$	4: at most 1 'assign' per iteration ($\leq$ n ops)
5.	**end if**	
6.	**end for**	
7.	**return** max	7: 'return' (1 op)

Time Complexity
Linear Search Algorithm: O(n)

Total is at most $1 + 3(n + 1) + 2n + 2 = 5n + 6 = O(n)$

1.	$i \leftarrow 0$	1: 'assign' (1 op)
2.	**while** $i < n$ and $x \neq a_i$	2: '<', '≠', 'and' at most n+1 times ($\leq 3(n + 1)$ ops)
3.	**do** $i \leftarrow i + 1$	3: 'add', 'assign' at most n times ($\leq 2n$ ops)
4.	**end while**	
5.	**if** $i < n$ **then**	5-8: 'compare', 'return' (2 ops)
6.	**return** i	
7.	**else**	
8.	**return** 0	
9.	**end if**	

Time Complexity
Binary search: O(log n)

Total is at most: $2 + (x + 1) + 4x + 4x + 3 = 9x + 6$. What is x?

1.		1: 'assign' (2 ops)
2.		2: 'compare' x + 1 times (x to be determined)
3.		3: 4 ops x times (4x ops)
4.		4-8: at most 4 ops x times (≤ 4x ops)
5.		10-11: (3 ops)
6.		
7.		
8.		
9.		
10.		
11.		

Behavior of Binary Search

How many iterations of the while loop are there?

n: # of elements in the list

- Assume n is a power of 2, $n = 2^k \rightarrow k = \log_2(n)$
- At each iteration, i increases or j decreases
- Iteration 1, $j - i = \frac{2^k}{2} = 2^{k-1}$
- Iteration 2, $j - i = \frac{2^{k-1}}{2} = 2^{k-2}$
- Iteration 3, $j - i = \frac{2^{k-2}}{2} = 2^{k-3}$
- How many iterations until $i \geq j$ (stopping condition of while)?
- After k iterations, $j - i = 2^0$, so after one more $i \geq j$
  Thus $x = k + 1 = \log_2(n) + 1$

Time Complexity
Optimized Insertion Sort $O(n^2)$

1.	**for** j ← 1 . . . n-1 **do**	1: test n times, each test takes 2 ops (3(n-1) ops)
2.	m ← A[j]	2: 2 ops per for-loop iteration (2(n - 1) ops)
3.	i ← j - 1	3: 2 ops per for-loop iteration (2(n - 1) ops)
4.	**while** (i >0) ∧ (A[i] > m)	4: test on a variable, x times. 4 ops per test.
5.	A[i + 1] ← A[i]	5: 4 ops per while-loop iteration
6.	i←i - 1	6: 2 ops per while-loop iteration
7.	**end while**	
8.	A[i + 1] ← m	8: 3 ops per for-loop iteration (3n ops)
9.	**end for**	
10.	**return** A	10: 1 op

Common Terminologies

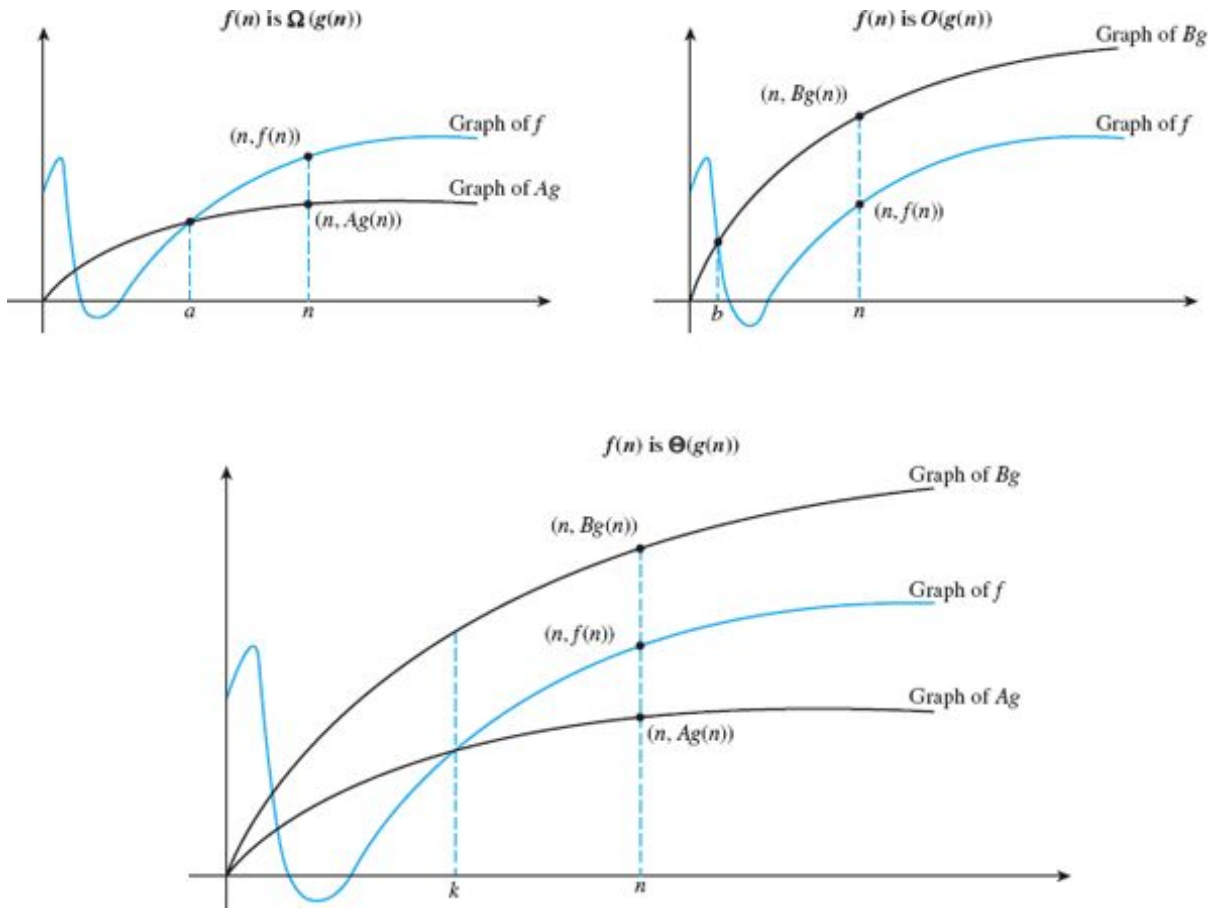
Constant Complexity: $O(1)$ Running time is independent of the input size e.g. return the first element of a list
Logarithmic Complexity: $O(\log n)$ e.g. binary search
Linear Complexity: $O(n)$ e.g. linear search
Quadratic Complexity: $O(n^2)$ e.g. insertion sort
Polynomial Complexity: $O(n^b)$ for some constant b e.g. all of the above
Exponential Complexity: $O(b^n)$ for some constant b e.g. generate the powerset of some input set of n elements

Upper and Lower Bounds

Big-Oh gives us upper bounds on functions: if f(x) is O(g(x)), then the value of f(x) is at most the value of g(x) (for large enough x)

If we want to know a lower bound on a function?
This is done with the "Big-Omega" notation !

Upper and Lower Bounds



Big-Omega Notation Ω()

$f(x)$ is Big-Omega of a function $g(x)$ if there are positive constants C and k such that $f(x) \geq C \cdot g(x)$ for all $x > k$.

Notation: $f(x) \in \Omega(g(x))$ or $f(x) = \Omega(g(x))$.

e.g.

$f(x) = 8x^3 + 5x^2 + 7$ is $\Omega(x^3)$. Verify with C=1 and k=1

note: $f(x)$ is $\Omega(g(x))$ if and only if $g(x)$ is $O(f(x))$.

Big-Omega Estimates (Witnesses C and k)

To find the witness C for a lower bound g(x)
Remove the lower order terms with positive coefficients.

Example: $f(x) = x^8 + 7x^7 - 10x^5 - 2x^4 + 3x^2 - 17$

$$\begin{aligned} x^8 + 7x^7 - 10x^5 - 2x^4 + 3x^2 - 17 &\geq x^8 - 10x^5 - 2x^4 - 17 \\ &\geq x^8 - 10x^7 - 2x^7 - 17x^7 \\ &\geq x^8 - 29x^7 \end{aligned}$$

Next, find a matching pair (C, k) that satisfies the inequality $x^8 - 29x^7 \geq Cx^8$.

$x^8 - 29x^7 \geq Cx^8 \rightarrow C \leq 1 - \frac{29}{x}$

If $x \geq 58$, then $C = \frac{1}{2}$ suffices.

$\therefore x^8 - 29x^7 \geq \frac{1}{2}x^8$ for $x \geq 58$

$\therefore x^8 + 7x^7 - 10x^5 - 2x^4 + 3x^2 - 17 \geq x^8 - 29x^7 \geq \frac{1}{2}x^8$

Big-Omega Example 1

Find a lower bound on n log(n) - 2n +13.

Lets find C such that: $C\, n \log(n) \leq n \log(n) - 2n + 13$ for $n \geq k$

Since $n \log(n) - 2n \leq n \log(n) - 2n + 13$

Let's find the coefficient C such that: $C\, n \log(n) \leq n \log(n) - 2n$

$C\, n \log(n) \leq n \log(n) - 2n \rightarrow C \leq 1 - \frac{2}{\log n}$, when $n > 1$.

for $n \geq 8$, $\frac{2}{\log n} \leq \frac{2}{3}$, and choosing $C = \frac{1}{3}$ suffices.

$C\, n \log(n) \leq n \log(n) - 2n \leq n \log(n) - 2n + 13$.

$\therefore$ f(n) is $\Omega(n \log(n))$ with the witnesses ($C = \frac{1}{3}$, $k = 8$)

Big-Omega Example 2

Find a lower bound on the sum of the first n positive integers.
Assume n is even.

$$f(n) = \sum_{k=1}^{n} k = 1 + 2 + 3 + \ldots + (n-1) + n$$

$$> \left(\frac{n}{2} + 1\right) + \left(\frac{n}{2} + 2\right) + \ldots + (n-1) + n$$

$$> \frac{n}{2} + \frac{n}{2} + \ldots + \frac{n}{2} + \frac{n}{2}$$

$$= \frac{n}{2} \cdot \frac{n}{2} = \frac{n^2}{4}$$

$\therefore$ f(n) is $\Omega(n^2)$ with the witnesses ($C = \frac{1}{4}$, $k = 1$)

Big-Theta Notation Θ()

What if a function f(x) is both O(g(x)) and Ω(g(x))?

f(x) is Big-Theta of function g(x) if:
f(x) is O(g(x)) and f(x) is Ω(g(x))

Notation: $f(x) \in \Theta(g(x))$ or $f(x) = \Theta(g(x))$

The behavior of f is lying between an upper bound and lower bound that are asymptotically equal. This is a tight bound. Informally, f "=" g.

Big-Theta Example 1

The sum of the first n positive integers is $O(n^2)$

The sum of the first n positive integers is $\Omega(n^2)$

Therefore, the sum of the first n positive integers is $\Theta(n^2)$

Big-Theta Example 2

Find a tight bound on $f(x) = x^4 - 23x^3 + 12x^2 + 15x - 21$.

$x^4 - 23x^3 + 12x^2 + 15x - 21 \leq (1+23+12+15+21)x^4 = 72x^4$ for $x \geq 1$

$\therefore x^4 - 23x^3 + 12x^2 + 15x - 21$ is $O(x^4)$ for $x \geq 1$

$$\begin{aligned} x^4 - 23x^3 + 12x^2 + 15x - 21 &\geq x^4 - 23x^3 - 21 \\ &\geq x^4 - 23x^3 - 21x^3 \\ &\geq x^4 - 44x^3 \end{aligned}$$

$x^4 - 44x^3 \geq C\, x^4$

$C \leq 1 - \frac{44}{x}$

If $x \geq 88$, then $C = \frac{1}{2}$ suffices; therefore, $x^4 - 44x^3 \geq \frac{1}{2} x^4$ for $x \geq 88$

$\therefore x^4 - 23x^3 + 12x^2 + 15x - 21 \geq x^4 - 44x^3 \geq \frac{1}{2} x^4$

$\therefore x^4 - 23x^3 + 12x^2 + 15x - 21$ is $\Omega(x^4)$ for $x \geq 88$

$\frac{1}{2} x^4 \leq x^4 - 23x^3 + 12x^2 + 15x - 21 \leq 72x^4$

$\therefore x^4 - 23x^3 + 12x^2 + 15x - 21$ is $\Theta(x^4)$

Practice

Is there a Big-Θ estimate for this function?

$f(x) = 4x \log(x^9) + 16x^2 + 3$

Search Algorithms Time Complexity

	Best	Worst
Linear search	$\Omega(1)$	$O(n)$
Binary search	$\Omega(1)$	$O(\log(n))$

Sorting Algorithms Time Complexity

	Best	Worst
Selection Sort	$\Omega(n^2)$	$O(n^2)$
Bubble Sort	$\Omega(n)$	$O(n^2)$
Insertion Sort	$\Omega(n)$	$O(n^2)$
Heap Sort	$\Omega(n \log(n))$	$O(n \log(n))$
Quick Sort	$\Omega(n \log(n))$	$O(n^2)$
Merge Sort	$\Omega(n \log(n))$	$O(n \log(n))$
Bucket Sort	$\Omega(n + k)$	$O(n^2)$
Radix Sort	$\Omega(nk)$	$O(nk)$